



Chapter 1 Introduction

1.1 GitHub

1) Personal GitHub Account

My personal GitHub account: <https://github.com/Cole-Koru>

2) Group Project Repository

Group project repository URL: <https://github.com/Chintsogt0825/Midterm>

This repository contains the Scrapy project used to extract metadata from private GitHub repositories using the GitHub API.

1.2 Overview

This project demonstrates advanced Python programming concepts and tools, such as:

- **dataclass**: For representing structured data
- **pprint**: For pretty-printing JSON and structured outputs
- **Regular Expressions (re)**: For string pattern matching

- **GitHub API:** For secure and structured access to GitHub data
- **Scrapy:** For building an asynchronous crawler
- **JSON and XML serialization:** For storing structured output
- **Authentication headers:** For accessing private repositories securely

The goal of the project is to extract the following information for each repository:

- **URL**
- **About:** The description, or repository name if the description is empty and the repo is not empty
- **Last Updated** date
- **Languages** used (via GitHub's language API)
- **Number of Commits**

Chapter 2: Implementation

2.1 Class 1: **GithubSpider**

This class extends `scrapy.Spider` and is the core of the scraping logic.

2.1.1 Fields

- **name:** Unique spider name (used in terminal)
- **allowed_domains:** Domain whitelist (`api.github.com`)
- **start_urls:** Starting point (`https://api.github.com/user/repos`)
- **token:** Personal Access Token for GitHub API authentication

2.1.2 Methods

`start_requests(self)`

- Sets up authentication headers
- Initiates the request to GitHub API for all user repositories

`parse(self, response)`

- Parses repository metadata
- If the description is empty but repo is not, sets repo name as `about`
- Schedules requests for commits and languages

`parse_commits(self, response)`

- Counts the number of commits in a repository by measuring the JSON response length
- Passes data to `parse_languages`

`parse_languages(self, response)`

- Fetches and extracts programming languages used in each repository
- Yields the final data object for each repository

2.2 Class 2

Not applicable – all logic is handled within a single spider class.

2.3 Function 1: `yield repo_data`

- At the end of the spider chain, this yields the final dictionary containing all the required fields.

2.4 Function 2: N/A

- All auxiliary logic is encapsulated in the spider methods.

Chapter 3: Results

3.1 Result 1 – Output File

The spider was run using:

```
scrapy crawl github_spider -o output.xml
```

It produced an XML file `output.xml` containing all required information:

```
<item>
  <url>https://github.com/your-username/my-repo</url>
  <about>My Project Repo</about>
  <last_updated>2025-04-10T12:30:00Z</last_updated>
  <languages>
    <language>Python</language>
    <language>HTML</language>
  </languages>
  <commits>15</commits>
</item>
```

3.2 Result 2 – Edge Cases

- Repositories with no description but not empty correctly defaulted `about` to repository name.
- Private repositories were accessed securely using a GitHub PAT.

Chapter 4: Conclusions

This project successfully demonstrates the use of Scrapy with the GitHub API to extract repository metadata. Key learning outcomes include:

- Use of authentication headers to access private resources
- Parsing and transforming JSON API responses
- Chaining Scrapy requests with metadata using `meta`
- Exporting structured data to XML

Improvements for future work:

- Implement pagination to support >30 repositories
- Handle rate-limiting dynamically
- Extend support to scrape contributors, forks, and stars